

SPRINT 3 BIG QUERY

YONA MANRIQUEZ

P2P ROSER CARRERA

Escenario

Acaba de llegar a la empresa. El equipo de Ingeniería de Datos ha dejado los archivos históricos en el Data Lake corporativo.

Tu primera misión es diseñar y crear el entorno de trabajo. El CTO exige profesionalidad: no volcaremos datos sin orden, sino que implementaremos una Arquitectura Medallion.

Nivel 1 Entorno e Ingesta Híbrida (Code-Firts)

Ex. 1: Arquitectura de Datos (Lógica vs Física)

Implementarás físicamente la arquitectura lógica creando tres Datasets dentro de un mismo proyecto para centralizar la facturación y los permisos.

Crea un Nuevo Proyecto en Google Cloud denominado: sprint3-analytics-[tu_nombre] (Lo puedes crear por la UI, pero si lo haces por Consola (gcloud CLI) tiene un plus).

Para demostrar tu dominio total de la plataforma, crearás cada uno de los datasets utilizando un método diferente:

- Dataset Físico sprint3_bronze → Capa Lógica Bronze (Datos Crudos)
- Función: Datos crudos tal como llegan de la fuente.
- Método: Utiliza la Interface Gráfica (UI) de BigQuery (haciendo clicks).
- Dataset Físico sprint3_silver → Capa Lógica Silver (Datos Limpios)
- Función: Datos limpios, tipados y duplicados.

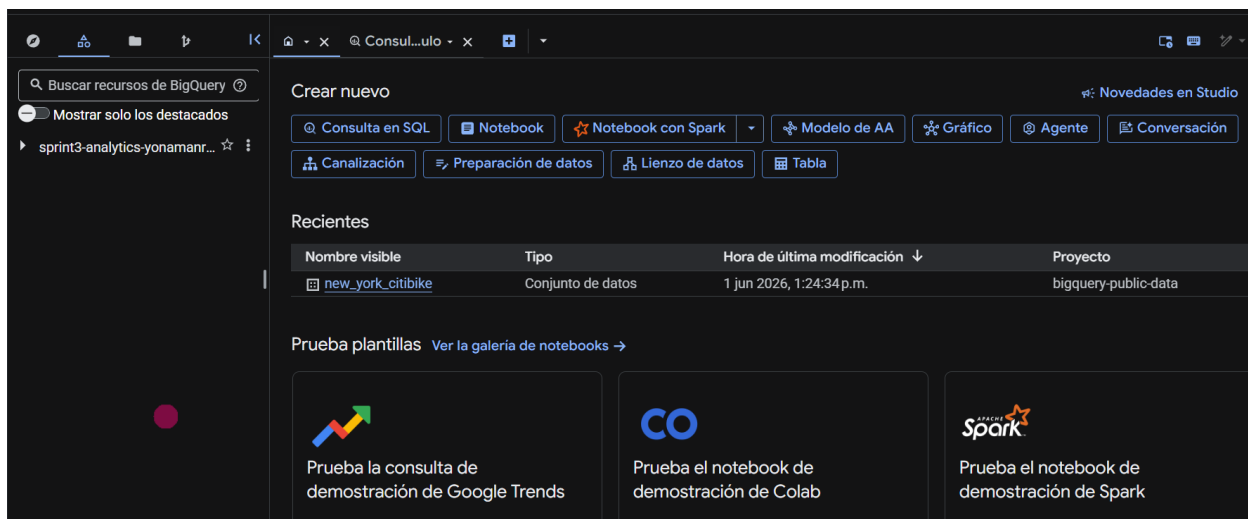
- Método: Utiliza código SQL (CREATE SCHEMA).
- Dataset Físico sprint3_gold → Capa Lógica Gold (Datos de Negocio)
- Función: Datos agregados listos para informes y paneles de control (dashboards).
- Método: Utiliza Cloud Shell (Línea de órdenes bq).

Ten en cuenta la ubicación de los datos originales para decidir la ubicación física de los tres Datasets.

Nota: Aunque sean capas lógicas diferentes, físicamente han de vivir en la misma región para poder "hablar" entre ellas sin costos de transferencia.

=====

Se crea el proyecto sprint3-analytics-yonamanrque mediante UI **User Interface**)



Creemos cada dataset.

La capa Broze se creó a partir de **UI (User Interface)**

Crear un conjunto de datos

ID del proyecto *
sprint3-analytics-yonamanrique Cambiar

ID de conjunto de datos *
sprint3_bronze

Puede incluir letras, números y guiones bajos

Ubicación de los datos
EU (múltiples regiones en la Unión Europea)

Conjunto de datos externo

The selected region supports the following external dataset types: Cloud Spanner

☐ Vínculo a un conjunto de datos externo

Etiquetas

Opciones avanzadas

Crear conjunto de datos Cancelar

Dataset Silver (sprint3_silver) a través de **código SQL** que crea un dataset en BigQuery "schema", el nombre completo sprint3-analytics-yonamanrique.sprint3_silver y la localización 'EU'

Consulta sin título

```
1 CREATE SCHEMA `sprint3-analytics-yonamanrique.sprint3_silver`
2 OPTIONS (
3   description = 'Silver: datos limpios',
4   location = 'EU'
5 );
```

Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico

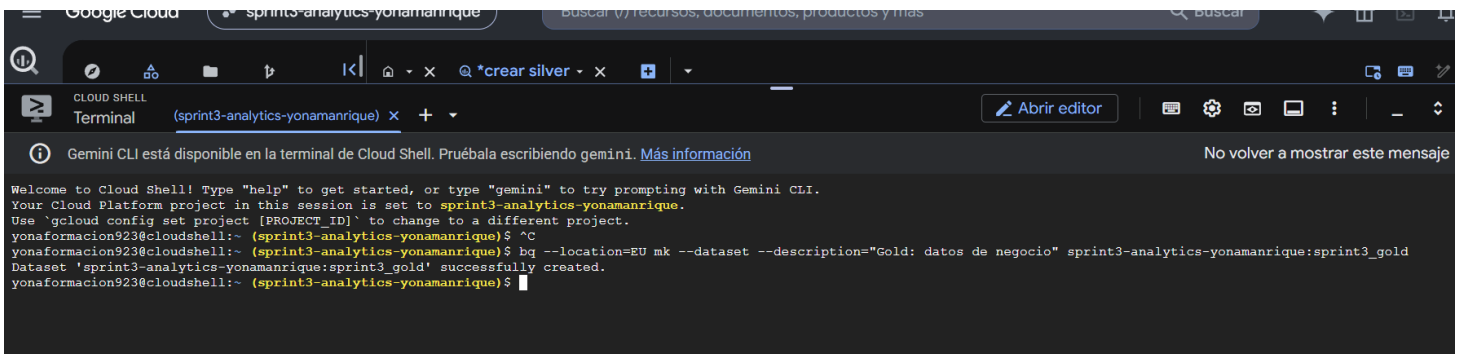
Se creó el conjunto de datos llamado sprint3_silver.

Dataset **Gold** creado a partir de **Cloud Shell** que son la líneas de comando. (Nota Importante dar a Autorizar)

Se escribe la sintaxis siguiente:

```
bq --location=EU mk --dataset --description="Gold: datos de negocio"
sprint3-analytics-yonamanrique:sprint3_gold
```

Esta sintaxis indica, la localización en Europa, igual que los anteriores, "mk", que es la de crear el dataset y añadimos la descripción. sprint3-analytics-yonamanrique:sprint3_gold



The screenshot shows a Google Cloud Shell terminal window. At the top, there's a header with the Google Cloud logo and the project name 'sprint3-analytics-yonamanrique'. Below the header, there's a terminal window titled 'Terminal' with a tab for 'sprint3-analytics-yonamanrique'. The terminal content shows a welcome message, instructions on how to use the shell, and the execution of the BigQuery command to create a dataset. The command is: `bq --location=EU mk --dataset --description="Gold: datos de negocio" sprint3-analytics-yonamanrique:sprint3_gold`. The output shows that the dataset 'sprint3-analytics-yonamanrique:sprint3_gold' was successfully created. The prompt is now `younaformacion923@cloudshell:~ (sprint3-analytics-yonamanrique)$`.

```
Google Cloud sprint3-analytics-yonamanrique
Terminal sprint3-analytics-yonamanrique x +
Gemini CLI está disponible en la terminal de Cloud Shell. Pruébala escribiendo gemini. Más información No volver a mostrar este mensaje

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to sprint3-analytics-yonamanrique.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
younaformacion923@cloudshell:~ (sprint3-analytics-yonamanrique)$ ^C
younaformacion923@cloudshell:~ (sprint3-analytics-yonamanrique)$ bq --location=EU mk --dataset --description="Gold: datos de negocio" sprint3-analytics-yonamanrique:sprint3_gold
Dataset 'sprint3-analytics-yonamanrique:sprint3_gold' successfully created.
younaformacion923@cloudshell:~ (sprint3-analytics-yonamanrique)$
```

EX 2. Ingesta en Capa Bronze (Conexion DDL)

Escribe y ejecuta las sentencias CREATE EXTERNAL TABLE para conectar los siguientes ficheros del dataset sprint3_bronze. Presta mucha atención a las "Notas Técnicas", ya que no todos los archivos tienen el mismo formato.

Taula	Domini	Ruta al Núvol (URI)	Notes Tècniques
transactions_raw	ERP	gs://bootcamp-data-analytics-public/ERP/transactions.csv	Delimitador: ; (Punt i coma)
companies_raw	ERP	gs://bootcamp-data-analytics-public/ERP/companies.csv	Capçalera: Ignora la 1a fila. Defineix l'esquema manualment perquè tot és text.
american_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/american_users.csv	Estàndard.
european_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/european_users.csv	Estàndard.
credit_cards_raw	CRM	gs://bootcamp-data-analytics-public/CRM/credit_cards.csv	Estàndard.

Entrega: el código SQL utilizado para transactions_raw y una captura de pantalla con las 5 tablas creadas.

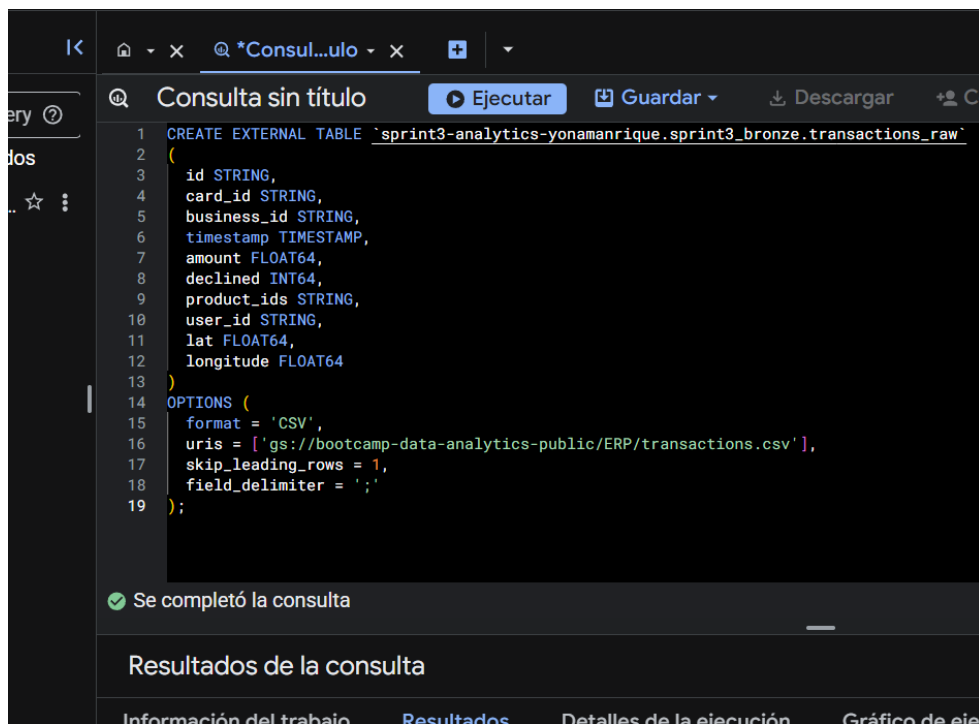
=====

Para poder consultar una tabla externa sin necesidad de importarlos, la sentencia CREATE EXTERNAL TABLE de SQL crea esa conexión. BigQuery los lee desde su ubicación original.

Para ver los datos de la tabla transaction se uso autodetect = TRUE sin embargo, aparecían errores de detección. Es por ello que se decide hacerlo manualmente desde las consultas.

Para crear las tablas se utilizo la sintaxis de Código de [https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language#create external table statement](https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language#create%20external%20table%20statement)

```
CREATE [ OR REPLACE ] EXTERNAL TABLE [ IF NOT EXISTS ] table_name
[(
    column_name column_schema,
    ...
)]
[WITH CONNECTION {connection_name | DEFAULT}]
[WITH PARTITION COLUMNS
    [(
        partition_column_name partition_column_type,
        ...
    )]
]
OPTIONS (
    external_table_option_list,
    ...
);
```



```
1 CREATE EXTERNAL TABLE `sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw`
2 (
3   id STRING,
4   card_id STRING,
5   business_id STRING,
6   timestamp TIMESTAMP,
7   amount FLOAT64,
8   declined INT64,
9   product_ids STRING,
10  user_id STRING,
11  lat FLOAT64,
12  longitude FLOAT64
13 )
14 OPTIONS (
15   format = 'CSV',
16   uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],
17   skip_leading_rows = 1,
18   field_delimiter = ';'
19 );
```

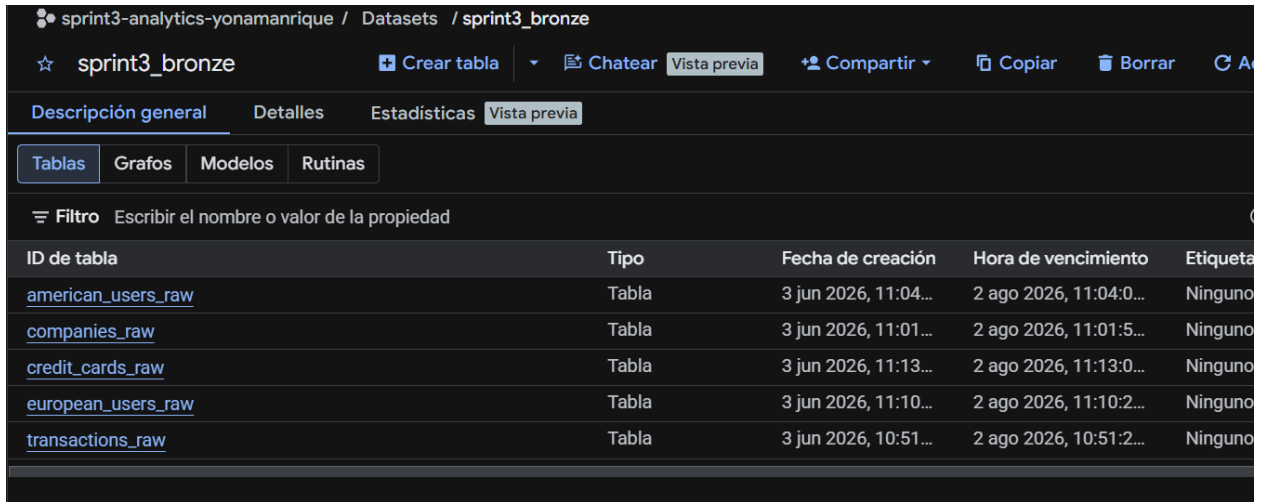
Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

Tabla `transaction_raw`

Imagen de las tablas creadas



The screenshot shows the Databricks workspace interface for a dataset named 'sprint3_bronze'. The 'Tablas' (Tables) tab is selected, displaying a list of five external tables. The interface includes navigation tabs for 'Descripción general', 'Detalles', 'Estadísticas', and 'Vista previa'. A filter bar is present above the table list. The table list has columns for 'ID de tabla', 'Tipo', 'Fecha de creación', 'Hora de vencimiento', and 'Etiqueta'.

ID de tabla	Tipo	Fecha de creación	Hora de vencimiento	Etiqueta
american_users_raw	Tabla	3 jun 2026, 11:04...	2 ago 2026, 11:04:0...	Ninguno
companies_raw	Tabla	3 jun 2026, 11:01...	2 ago 2026, 11:01:5...	Ninguno
credit_cards_raw	Tabla	3 jun 2026, 11:13...	2 ago 2026, 11:13:0...	Ninguno
european_users_raw	Tabla	3 jun 2026, 11:10...	2 ago 2026, 11:10:2...	Ninguno
transactions_raw	Tabla	3 jun 2026, 10:51...	2 ago 2026, 10:51:2...	Ninguno

Se han creado las cinco tablas externas mediante el código `CREATE EXTERNAL TABLE`, especificando manualmente el esquema (columna por columna y su tipo de dato), no con autodetect por lo explicado anteriormente.

Ex. 3 Carga de Datos Locales (Upload)

Falta el catálogo de Productos. Esta información no está en el Data Lake, utiliza el fichero products.csv del sprint 2.

- Carga este fichero manualmente ("Upload") en el dataset sprint3_bronze y crea la tabla products_raw.
- Observa que esta será la única Tabla Nativa de momento.

=====

Para este ejercicio usaremos la UI de BigQuery para subir nuestra archivo. Importante para acceder hay que ir al sprint3_bronze y crear tabla.

Crear tabla

Fuente

Create table from
Subir

Seleccionar archivo *
N1-Ex.8__products.csv

Formato del archivo
CSV

Destino

Proyecto *
sprint3-analytics-yonamanrique

Conjunto de datos *
sprint3_bronze

Tabla *
products_raw

El tamaño máximo del nombre es de 1,024 bytes UTF-8. Se permiten letras, signos, números, conectores, guiones y espacios Unicode.

Tipo de tabla
Tabla nativa

Mostrar solo los destacados

sprint3-analytics-yonam...

Consultas

Consultas (clásicas) (1)

Notebooks

Lienzos de datos

Preparaciones de datos

Canalizaciones

sprint3_bronze

american_users_r

companies_raw

credit_cards_raw

european_users_r

products_raw

transactions_raw

sprint3-analytics-yonamanrique / Datasets / sprint3_bronze

sprint3_bro...

Crear tabla

Chatear

Vista previa

Descripción general

Detalles

Estadísticas

Vista previa

Tablas

Grafos

Modelos

Rutinas

Filtro

Escribir el nombre o valor de la propiedad

ID de tabla	Tipo
american_users_raw	Tabla
companies_raw	Tabla
credit_cards_raw	Tabla
european_users_raw	Tabla
products_raw	Tabla
transactions_raw	Tabla

Esquema de la tabla products_raw

☆	products_raw	Consulta	Abrir en	Compartir				
<	Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje	Pe
☐	Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valor predete	
☐	id	INTEGER	NULLABLE	-	-	-	-	
☐	product_name	STRING	NULLABLE	-	-	-	-	
☐	price	FLOAT	NULLABLE	-	-	-	-	
☐	colour	STRING	NULLABLE	-	-	-	-	
☐	weight	FLOAT	NULLABLE	-	-	-	-	
☐	warehouse_id	STRING	NULLABLE	-	-	-	-	
☐	category	STRING	NULLABLE	-	-	-	-	
☐	brand	STRING	NULLABLE	-	-	-	-	
☐	cost	FLOAT	NULLABLE	-	-	-	-	
☐	launch_date	DATE	NULLABLE	-	-	-	-	

Ex. 4: Arquitectura y Rendimiento.

Materialización de Datos (Asistido por IA)

Escenario:

Las consultas sobre el Data Lake van lentas. Debes demostrar a tu manager la diferencia de rendimiento entre trabajar con ficheros externos y trabajar con tablas nativas de BigQuery.

a) Materialización de Datos (Asistido por IA)

Tarea: Crea una nueva tabla denominada `sprint3_bronze.transactions_raw_native` que sea una copia exacta de tu tabla externa `transactions_raw`.

- Requisito de "AI-Pair Programming": No escribas el código SQL manualmente. Utiliza el Asistente de IA (Gemini) integrado a BigQuery (o en el chat lateral) para generar la sentencia.
- Tu rol: Eres el responsable de validar que el código que te da la IA es correcto antes de ejecutarlo.
- Prompt Sugerido: "Write a SQL query to create a new table called `transactions_raw_native` in the `sprint3_bronze` dataset. It should contain all data from the `transactions_raw_table`. Please use `CREATE OR REPLACE TABLE` so I don't get errors if I run it more than once."
- Entrega: una captura de pantalla donde se vea:

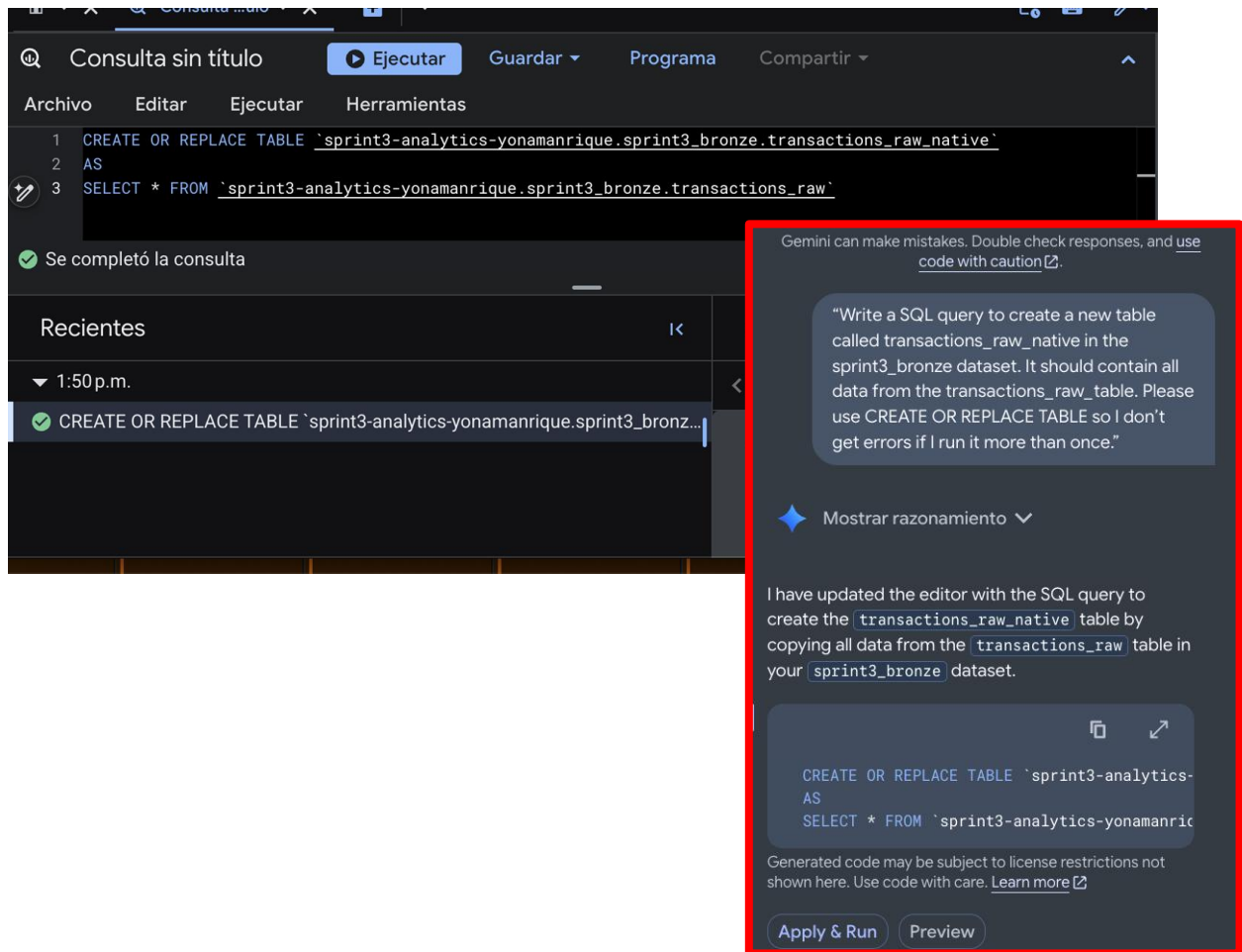
El prompt que escribiste a la IA.

El código SQL que la IA generó.

El resultado de la ejecución ("This statement created a new table...").

=====

Desde la creación de las tablas en los ejercicios anteriores se colocó `skip_leading_rows = 1` que significa "salta la primera fila ya que normalmente en un csv es la cabecera y viendo la estructura de los datos del `sprint2`, seguramente estos tendrían la misma estructura. De esta forma el código que nos da Gemini no tuvo problemas al ejecutarse.



b) Auditoría de Costos

Tarea: Diseñar una prueba para comparar el costo de leer una sola columna (ex. id) en la tabla externa vs. la tabla nativa. Utiliza los Bytes procesados y los Bytes facturados para hacer la comparación.

- ¿Cuánto "pesa" la consulta a transactions_raw?
- ¿Cuánto "pesa" la misma consulta en transactions_raw_native?

Entrega: Capturas del validador mostrando la diferencia y una conclusión técnica.

Archivo Editar Ejecutar Herramientas

```

1 SELECT id
2 FROM `sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw`;

```

Se completó la consulta

Recientes ⋮ Resultados de la consulta Chat View all jobs Guardar los resulta

▼ 1:57 p.m. Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico de ejecución

✓ SELECT id FROM `sprin...

ID de trabajo	sprint3-analytics-yonamanrique:EU.bquxjob_428f0f1d_19ec0d7f943 (Ver detalles de rendimiento)
Usuario	yonamanriquez@gmail.com
Ubicación	EU
Hora de creación	13 jun 2026, 1:57:17 p.m. UTC+2
Hora de inicio	13 jun 2026, 1:57:18 p.m. UTC+2
Hora de finalización	13 jun 2026, 1:57:19 p.m. UTC+2
Duración	1 s
Bytes procesados	12.61 MB
Bytes facturados	13 MB
Milisegundos de ranura	1022
Prioridad del trabajo	INTERACTIVE
Usar SQL heredado	false

Tabla externa transactions_raw: 12.61 MB procesados, 13 MB facturados

Consulta sin título Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

```

1 SELECT id
2 FROM `sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw_native`;

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes ⏪ Resultados de la consulta Chat View all jobs Guardar los resulta

▼ 2:00 p.m. Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico de ejecución

✓ SELECT id FROM `sprin...

ID de trabajo	sprint3-analytics-yonamanrique:EU.bquxjob_1f29f63c_19ec0da9ab1 (Ver detalles de rendimiento)
Usuario	yonamanriquez@gmail.com
Ubicación	EU
Hora de creación	13 jun 2026, 2:00:10 p.m. UTC+2
Hora de inicio	13 jun 2026, 2:00:10 p.m. UTC+2
Hora de finalización	13 jun 2026, 2:00:11 p.m. UTC+2
Duración	1 s
Bytes procesados	3.62 MB
Bytes facturados	10 MB
Milisegundos de ranura	893

Tabla nativa: 3.62 MB procesados, 10 MB facturados

Si realizamos una división simple de los datos procesados: $12.61/3.62$ se encuentra que la tabla nativa procesó 3.5 menos datos que la tabla externa, con la misma consulta.

Al comparar el procesamiento de una misma consulta sobre la tabla externa y su versión nativa, se observa una diferencia notable como se ha mencionado.

De acuerdo a la documentación de google cloud⁶ esto ocurre porque una tabla externa apunta directamente a un fichero CSV en Cloud Storage, que es un formato de texto plano sin ninguna optimización; por lo tanto, para extraer una sola columna, BigQuery necesita leer el fichero completo línea por línea.

En otro documento de Google Cloud⁵, se menciona que al materializar los datos como tabla nativa, BigQuery los almacena en su propio formato columnar interno, lo que le permite acceder únicamente a la columna solicitada sin tocar el resto.

Como conclusión, trabajar con tablas nativas no solo mejora el rendimiento de las consultas, sino que también reduce directamente el coste asociado.

c) El peligro del LIMIT

Muchos analistas piensan que poner LIMIT 10 reduce el costo de la consulta.

Tarea: Haz un `SELECT * FROM transactions_raw_native LIMIT 10` sobre la tabla externa.

- Comprobación: Mira el "Dry Run". ¿Ha bajado el costo respecto a no poner LIMIT? Explica brevemente por qué eso es una trampa para principiantes.

=====

Se piensa que al colocar Limit 10 solo se leen 10 filas y por lo tanto el coste es menor. Sin embargo, BQ lee columnas completas antes que devuelva el resultado, porque su motor es columnar, veo todos los valores de la columna y después filtra o limita, precisamente, es lo que se menciona en su página oficial:

"En el caso de las tablas no agrupadas en clústeres, aplicar una cláusula LIMIT a una consulta no afecta la cantidad de datos leídos. Se te cobra por leer todos los bytes en la tabla completa según lo indica la consulta, aunque esta solo muestre un subconjunto." ⁴.

Se entiende que si hacemos un `SELECT *` estamos aumentando los costes al no seleccionar solo las columnas a visualizar por lo que la literatura recomienda usar filtros WHERE para reducir filas.

"Con una tabla agrupada en clústeres, una cláusula LIMIT puede reducir la cantidad de bytes analizados, ya que el análisis se detiene cuando se analizan suficientes bloques para obtener el resultado. Solo se te cobrarán los bytes analizados"⁴

```
1 SELECT *
2 FROM 'sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw_native'
3 LIMIT 10;
```

Esta consulta procesará 10.65 MB cuando se ejecute.

```
1 SELECT *
2 FROM 'sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw_native';
3
```

Esta consulta procesará 10.65 MB cuando se ejecute.

Si se observan las dos imágenes, se observa que el limit no es un factor que altere los valores de procesamiento, las dos consultas da el mismo valor de 10.65 MB.

Para reducir costes, la solución correcta es seleccionar solo las columnas necesarias en vez de SELECT , y usar filtros WHERE para reducir filas.

Ex. 5: Adaptación de Sintaxis (Reporting)

Tu jefe quiere saber cuáles fueron los 5 días con más ingresos del año 2021.

- Reto: Probablemente el campo timestamp es un STRING. Tendrás que investigar funciones de BigQuery (SUBSTR, CAST, PARSE_TIMESTAMP) para filtrar el año y agrupar por fecha correctamente.

=====

La tabla creada desde el inicio mantuvo el tipo TIMESTAMP al diseñar el esquema de transactions, es por ello que no se usaron las funciones sugeridas como CAST para convertir a otro dato, ni PARSE_TIMESTAMP para convertir texto a fecha, sino que directamente se usó EXTRACT.

The screenshot shows the Google Cloud BigQuery console interface. At the top, there's a toolbar with buttons for 'Ejecutar' (Execute), 'Guardar' (Save), 'Programa' (Program), and 'Compartir' (Share). Below the toolbar is a menu bar with 'Archivo', 'Editar', 'Ejecutar', and 'Herramientas'. The main area displays a SQL query:

```
1 SELECT
2   DATE(timestamp) AS fecha,
3   ROUND(SUM(amount), 2) AS total_vendido
4 FROM `sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw_native`
5 WHERE EXTRACT(YEAR FROM timestamp) = 2021
6 GROUP BY fecha
7 ORDER BY total_vendido DESC
8 LIMIT 5;
```

Below the query, a status message indicates 'Se completó la consulta' (Query completed) and 'Se está usando la cuota de procesamiento según demanda' (Using processing quota as demand). The bottom section shows the 'Resultados de la consulta' (Query results) table. The table has columns for 'Fila' (Row), 'fecha' (Date), and 'total_vendido' (Total sold). The results are as follows:

Fila	fecha	total_vendido
1	2021-10-27	11440.81
2	2021-01-22	11328.84
3	2021-02-11	11182.16
4	2021-04-29	11119.01
5	2021-03-27	10898.51

Ex. 6: Consultas Complejas

Necesitamos un informe que cruce datos

Tarea: Lista el nombre, país y fecha de las transacciones realizadas por empresas que hicieron operaciones entre 100 y 200 euros en alguna de estas fechas: 29-04-2015, 20-07-2018 o 13-03-2024.

=====

Consulta_compleja

Ejecutar

Guardar

Programa

Comparar

Archivo

Editar

Ejecutar

Herramientas

1 SELECT

2 c.company_name AS nombre_empresa,

3 c.country AS pais,

4 DATE(t.timestamp) AS fecha_transaccion,

5 ROUND(t.amount, 2) AS monto

6 FROM 'sprint3-analytics-yonamanrique.sprint3-bronze.companies_raw' c

7 JOIN 'sprint3-analytics-yonamanrique.sprint3-bronze.transactions_raw' t

8 ON c.company_id = t.business_id

9 WHERE t.amount BETWEEN 100 AND 200

10 AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')

11 ORDER BY t.amount DESC;

Esta consulta procesará 0 B cuando se ejecute.

Recientes

Resultados de la consulta

Chat

View all jobs

Guardar los resultados

Abrir en

2:23 p.m.

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico de ejecución

SELECT c.company_na...

fila	nombre_empresa	pais	fecha_transaccion	monto
1	Donec Fringilla PC	France	2015-04-29	187.52
2	Netus Et Malesuada Ltd	Netherlands	2018-07-20	183.64
3	Auctor Mauris Vel LLP	United States	2015-04-29	181.42
4	Non Magna LLC	United Kingdom	2015-04-29	178.1

Resultados por página: 50

1 - 19 de 19

<<

<

>

>>

Aparecen 19 registros, lo que quiere decir que en la consulta se está ejecutando correctamente DATE como fecha para poder filtrar las transacciones por monto

.

NIVEL 2 Limpieza y Transformación (ELT)

Los datos brutos tienen problemas de calidad (símbolos de moneda, usuarios duplicados, fechas incorrectas). Actúa como Data Engineer para limpiarlo todo.

Ex. 1: Limpieza de Productos (Data Quality)

Crearemos la capa limpia ("Silver") para los productos. Tienes la tabla `sprint3_bronze.products_raw` cargada desde el CSV. El equipo de Data Governance te pide crear una tabla de productos limpia en `sprint3_silver.products_clean` que cumpla estas reglas de calidad:

- Estandarización de nombres: La columna original `id` es ambigua; renómbrala como `product_id`. La columna `product_name` simplifícala a `name`.
- Limpieza de IDs: El campo `warehouse_id` tiene el formato antiguo "WH-4". Elimina el prefijo "WH-" y convierte en valor a número entero (INT64).
- Garantía de Precio: Asegúrate que `price` es un número (FLOAT64), sin símbolos de moneda.
- Otras columnas: Conserva el campo `weight` (peso) tal cual.

=====

Se crea la tabla limpia en Silver, procurando que si ya existe, se remplace después del AS.

Se utiliza CAST para convertir la columna `id` a STRING renombrandolo como `product_id`, así no es tan genérico el nombre solo de "id".

Después extraemos los números de WH-4 con REGEXP_EXTRACT, pero esto nos devuelve un texto "4", por lo que después se convierte a número entero con el CAST e INT64.

Para el precio, primero se convierte a texto (CAST) para poder limpiar y eliminar los símbolos (REGEXP_REPLACE), espacios, letras y dejar solo los números. Acto seguido, volvemos a convertir este texto en número decimal. (CAST, FLOAT64)

```

1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_silver.products_clean`
2 AS
3 SELECT
4   CAST(id AS STRING) AS product_id,
5   product_name AS name,
6   colour,
7   weight,
8   CAST(REGEXP_EXTRACT(warehouse_id, r'(\d+)') AS INT64) AS warehouse_id,
9   CAST(REGEXP_REPLACE(CAST(price AS STRING), r'^0-9.', '') AS FLOAT64) AS price
10  FROM `sprint3-analytics-yonamanrique.sprint3_bronze.products_raw`;

```

Se completó la consulta

La tabla product_clean queda de la siguiente forma:

☆ products_cl... 🔍 Consulta 💬 Chat 📄 Abrir en ▾

< Esquema Detalles Vista previa Explorador de tablas Vi

descripciones.

Descartar Generar

🔍 Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción
☐	product_id	STRING	NULLABLE	-
☐	name	STRING	NULLABLE	-
☐	colour	STRING	NULLABLE	-
☐	weight	FLOAT	NULLABLE	-
☐	warehouse_id	INTEGER	NULLABLE	-
☐	price	FLOAT	NULLABLE	-

Ex. 2: Creación de Transacciones Limpias (Capa Silver)

Escenario:

La tabla `transactions_raw` (Bronze) es insegura: el importe (`amount`) podría tener letras, las coordenadas podrían fallar y la fecha es un texto difícil de filtrar. Crearemos la tabla definitiva `sprint3_silver.transactions_clean` que garantice lo siguiente:

- Estandarización de nombres: La columna original `id` es ambigua; **renómbrala como `transaction_id`**.
- Robustez en Imports: Utiliza **`SAFE_CAST`** en el campo `amount`. Si falla, sustitúyelo por 0 (usando `IFNULL`).
- Fechas Reales: Convierte el campo `timestamp` (que es `STRING`) a tipo `TIMESTAMP` real.
- Coordenadas: Asegura que **`lat` y `longitude` sean `FLOAT64`** (utiliza `SAFE_CAST` por seguridad).
- Desglose de Productos: Transforma la cadena de texto **`product_ids` en un `ARRAY`** de enteros.
- Input: `"1, 2, 3"` (`String`)
- Output: `[1, 2, 3]` (`Array` / Lista de enteros)

Resto de campos: Mantenlos igual

=====

Se cambia con `AS` el nombre de la columna `id` a `transaction_id`.

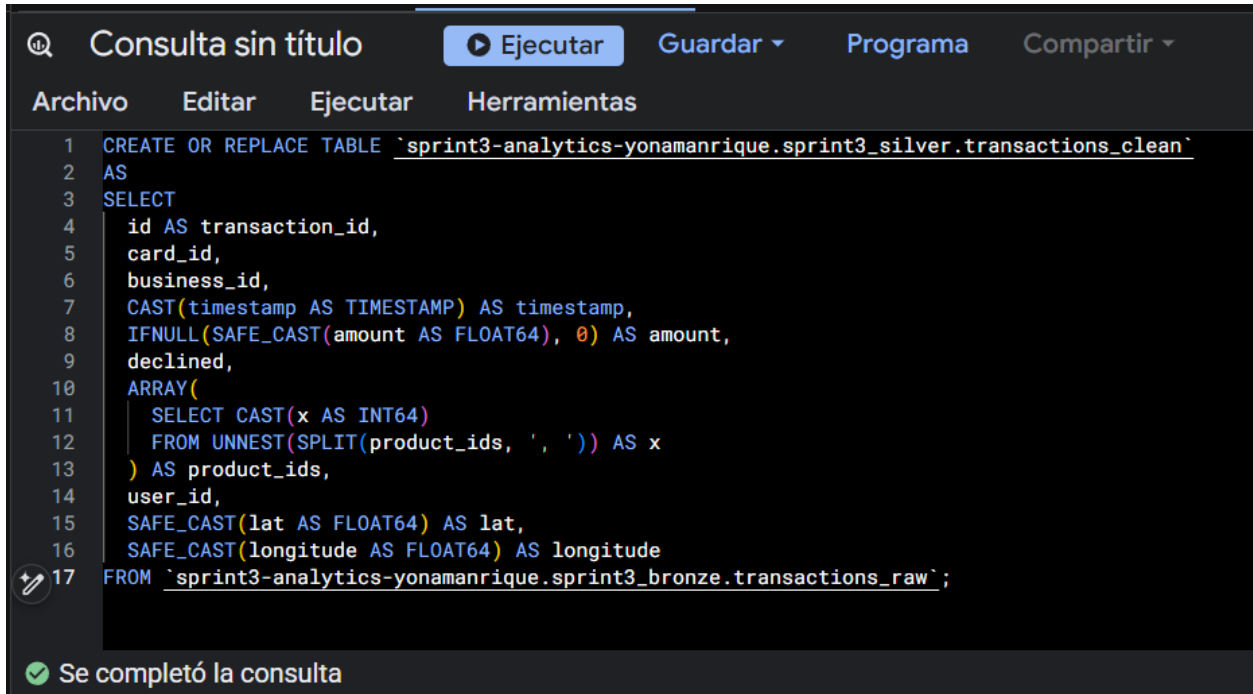
Se usa `CAST` para obtener el dato de tiempo por si hubiese un `STRING`, lo convierta.

`SAFE_CAST` ayuda a convertir `amount` en un número decimal sin dar error, sin embargo, si esto falla, la consulta debe devolver un `NULL`. Como el ejercicio solicita que los `NULL` cambien por cero, es por ello que se utiliza el `IFNULL`.

En este ejercicio se usa `SPLIT` para convertir los valores de `product_id` en lista. En bronze estos valores son un `STRING` continuo es

por ello que en Silver, necesitamos que los lea por separados, por eso se añade a la consulta", ".

Usamos UNEST para abrir ese array y se visualiza cada elemento por separado. con CAST(x AS INT64) convertimos cada texto en un número y con ARRAY se vuelve a hacer la lista, pero ahora numérica.



The screenshot shows a SQL query editor with a dark theme. At the top, there's a title bar with a magnifying glass icon, the text "Consulta sin título", and buttons for "Ejecutar", "Guardar", "Programa", and "Compartir". Below the title bar is a menu bar with "Archivo", "Editar", "Ejecutar", and "Herramientas". The main area contains a SQL query with line numbers 1 through 17. The query creates or replaces a table named 'transactions_clean' and selects data from 'transactions_raw'. It includes various casts and array operations. At the bottom, a green checkmark icon and the text "Se completó la consulta" indicate successful execution.

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_silver.transactions_clean`
2 AS
3 SELECT
4   id AS transaction_id,
5   card_id,
6   business_id,
7   CAST(timestamp AS TIMESTAMP) AS timestamp,
8   IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
9   declined,
10  ARRAY(
11    SELECT CAST(x AS INT64)
12    FROM UNNEST(SPLIT(product_ids, ',')) AS x
13  ) AS product_ids,
14  user_id,
15  SAFE_CAST(lat AS FLOAT64) AS lat,
16  SAFE_CAST(longitude AS FLOAT64) AS longitude
17 FROM `sprint3-analytics-yonamanrique.sprint3_bronze.transactions_raw`;
```

Se completó la consulta

Ex. 3: Unificación de Usuarios (UNION)

Tenemos los usuarios fragmentados por región.

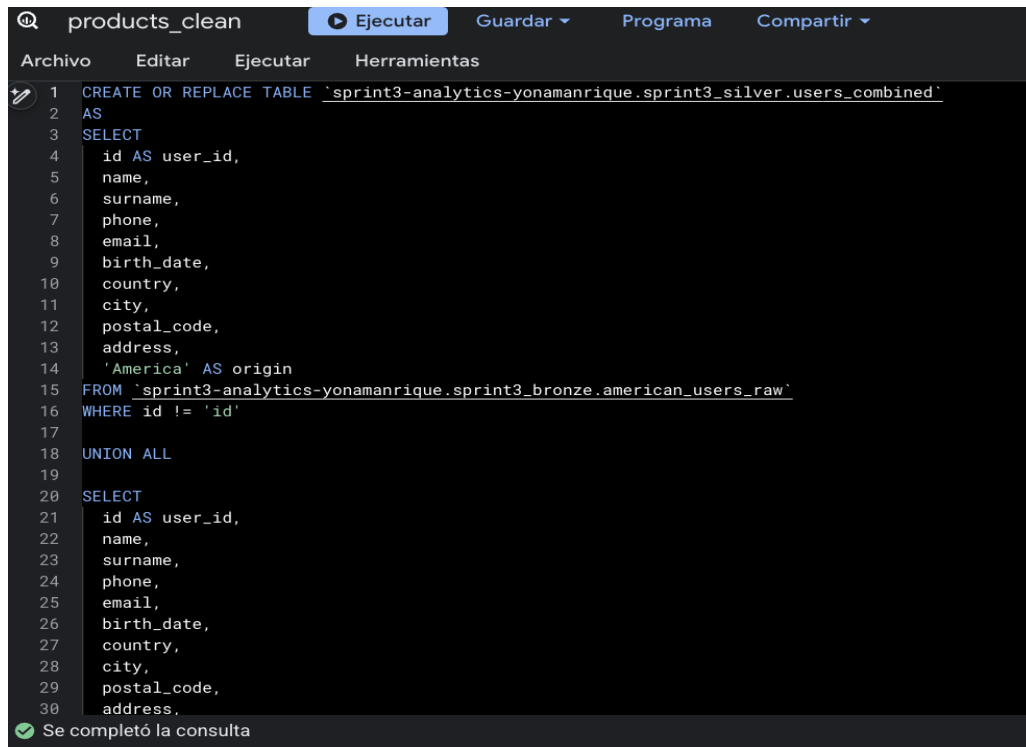
Tarea: Crea la tabla `sprint3_silver.users_combined`. Utiliza `UNION ALL` para unificar los usuarios de EEUU y Europa en una única lista maestra. Añade una columna calculada `origin` para saber de dónde vienen.

- Estandarización de nombres: La columna original `id` es ambigua; renómbrala como `user_id`.

=====

Como el ejercicio lo solicita, se unen las dos tablas de usuarios con `UNION ALL` creando nueva columna `origin` que se nombró como `'America'`, `'Europe'`.

Se añade `WHERE id != 'id'` debido a que los CSV originales tenían los nombres de las columnas en la primera fila, como se ha creado la tabla externa esa fila se ha colocado como si fuera un dato real, con ello descartamos cualquier `id` que se comporte como texto.



```
products_clean Ejecutar Guardar Programa Compartir
Archivo Editar Ejecutar Herramientas
1 CREATE OR REPLACE TABLE 'sprint3-analytics-yonamanrique.sprint3_silver.users_combined'
2 AS
3 SELECT
4     id AS user_id,
5     name,
6     surname,
7     phone,
8     email,
9     birth_date,
10    country,
11    city,
12    postal_code,
13    address,
14    'America' AS origin
15 FROM 'sprint3-analytics-yonamanrique.sprint3_bronze.american_users_raw'
16 WHERE id != 'id'
17
18 UNION ALL
19
20 SELECT
21     id AS user_id,
22     name,
23     surname,
24     phone,
25     email,
26     birth_date,
27     country,
28     city,
29     postal_code,
30     address,
```

Se completó la consulta

Ex. 4: Materialización de Compañías y Tarjetas de Crédito

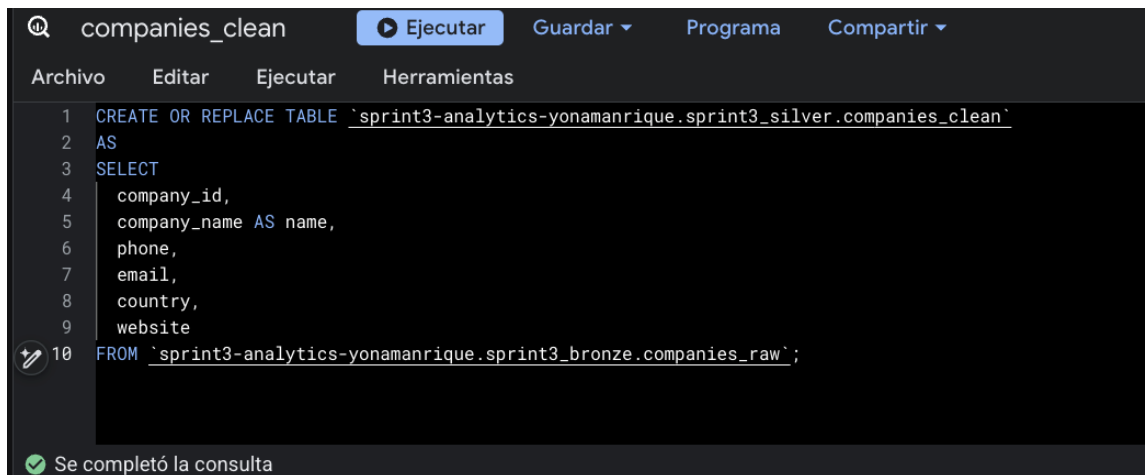
Escenario: Para completar el modelo de datos en la capa Silver, nos faltan dos dimensiones clave que actualmente dependen de ficheros CSV externos: Compañías y Tarjetas de Crédito. No podemos depender de ficheros sueltos para el análisis final. Hemos de importar estos datos en tablas nativas (Silver) y aprovechar para corregir formatos.

Tareas:

1. Crea la tabla `sprint3_silver.companies_clean`. Copia los datos tal cual, pero asegúrate que sea una tabla nativa de BigQuery (no una vista externa).
2. Crea la tabla `sprint3_silver.credit_cards_clean`. Copia los datos tal cual, pero asegúrate que sea una tabla nativa de BigQuery (no una vista externa).

Si hace falta, renombrar el id según corresponda.

=====



```
companies_clean  Ejecutar  Guardar  Programa  Compartir
Archivo  Editar  Ejecutar  Herramientas
1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_silver.companies_clean`
2 AS
3 SELECT
4   company_id,
5   company_name AS name,
6   phone,
7   email,
8   country,
9   website
10 FROM `sprint3-analytics-yonamanrique.sprint3_bronze.companies_raw`;

Se completó la consulta
```

```
credit_cards_clean  Ejecutar  Guardar  Programa  Compartir
Archivo  Editar  Ejecutar  Herramientas
1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_silver.credit_cards_clean`
2 AS
3 SELECT
4   id AS credit_card_id,
5   user_id,
6   iban,
7   pan,
8   pin,
9   cvv,
10  track1,
11  track2,
12  expiring_date
13 FROM `sprint3-analytics-yonamanrique.sprint3_bronze.credit_cards_raw`;

Se completó la consulta
```

```
Consulta sin titulo  Ejecutar  Guardar  Programa  Compartir
Archivo  Editar  Ejecutar  Herramientas
1 SELECT table_name
2 FROM `sprint3-analytics-yonamanrique.sprint3_silver.INFORMATION_SCHEMA.TABLES`;

Se completó la consulta
Se está usando la cuota de procesamiento según demanda
```

Recientes		Resultados de la consulta	
3:37p.m.		Información del trabajo	Resultados
SELECT table_name FR...	Fila	table_name	
	1	products_clean	
	2	credit_cards_clean	
	3	users_combined	
	4	transactions_clean	
	5	companies_clean	

Al final se han creado cinco tablas en Silver.

En este nivel se ha pasado de tener datos tal cual como llegaron a tener datos con los que realmente se puede trabajar.

La idea principal es que no se tocan los datos originales en Bronze, sino que se crean copias limpias en Silver. Así, si algo sale mal, siempre se puede volver al principio.

En products se tuvo que lidiar con precios que tenían el símbolo \$ y con IDs de almacén en formato "WH-4". En transactions se usó `SAFE_CAST` para protegernos de posibles datos corruptos, y convertimos una cadena de texto con IDs de productos en un array real para poder trabajar con ellos después.

En users se unieron dos tablas de regiones distintas en una sola lista, añadiendo una columna para saber de dónde venía cada usuario.

Al final se tienen 5 tablas limpias, bien nombradas y con los tipos de datos correctos.

Nivel 3: Presentación de Datos y Creación de Vistas

Ex. 1: La Vista de Marketing (Lógica de Negocio)

Tarea: Crea una vista denominada `sprint3_gold.v_marketing_kpis` que muestre la siguiente información para cada compañía.

- Nombre de la compañía, Teléfono y País (origen: `companies_clean`).
- Media de compra (`AVG(amount)` de `transactions_clean`).
- Clasificación de Cliente (Lógica):
- Crea una columna calculada denominada `client_tier`.
- Si la media de compra es superior a 260€, etiqueta como "Premium".
- Si es igual o inferior, etiqueta como "Standard".
- Entrega: Realiza una consulta `SELECT * FROM` sobre tu nueva vista, ordenando los resultados para que aparezcan primero los clientes "Premium" y, dentro de ellos, los que tengan mayor media de compra.

=====

Las vistas son consultas guardadas que se ejecutan cada vez que se llama. Se generan las vistas porque los KPIs de marketing cambian constantemente. Cada vez que hay transacciones nuevas, la media cambia. Con una vista siempre tienes los datos actualizados sin tener que recrear la tabla⁵

Se aplica un CASE para aplicar las condiciones de la media que pide el ejercicio. Y unimos con un LEFT JOIN para que se visualicen las empresas que no tengan transacciones

Después aplicamos el `SELECT * FROM` sobre la vista. Nos mostrará primero los clientes Premium y los de arriba del todo de la lista serán aquellos con un mayor promedio de compra.

gold_v_marketing_kpis Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

```

1 CREATE OR REPLACE VIEW `sprint3-analytics-yonamanrique.sprint3_gold.v_marketing_kpis`
2 AS
3 SELECT
4   c.name,
5   c.phone,
6   c.country,
7   ROUND(AVG(t.amount), 2) AS avg_amount,
8   CASE
9     WHEN AVG(t.amount) > 260 THEN 'Premium'
10    ELSE 'Standard'
11  END AS client_tier
12 FROM `sprint3-analytics-yonamanrique.sprint3_silver.companies_clean` c
13 LEFT JOIN `sprint3-analytics-yonamanrique.sprint3_silver.transactions_clean` t
14   ON c.company_id = t.business_id
15 GROUP BY c.name, c.phone, c.country;

```

Se completó la consulta

Consulta sin título Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

```

1 SELECT *
2 FROM `sprint3-analytics-yonamanrique.sprint3_gold.v_marketing_kpis`
3 ORDER BY avg_amount DESC;
4
5

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes	Resultados de la consulta					Chat	View all jobs	Guardar los resultados
3:46p.m.	Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución		
SELECT * FROM `sprint...	Fila	name	phone	country	avg_amount	client_tier		
	1	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87	Premium		
	2	Pretium Neque Corp.	07 77 48 55 28	Australia	276.16	Premium		
	3	Urna Convallis Associates	06 01 24 77 04	United States	274.24	Premium		
	4	At Associates	09 56 61 10 65	New Zealand	272.21	Premium		
	5	Metus Vitae Associates	08 25 44 40 66	Australia	270.08	Premium		
	6	Aliquet Diam Limited	02 76 61 47 46	United States	269.6	Premium		
	7	Nec Luctus LLC	02 14 71 75 73	Norway	268.6	Premium		

Resultados por página: 50 1 – 50 de 100

Ex. 2: Ranking de Productos (La Potencia de los Arrays)

Escenario de Negocio: El equipo de ventas quiere optimizar el catálogo. Necesitamos un informe en la capa Gold que muestre el rendimiento real de cada producto. Gracias a tu buena ingeniería en la capa Silver, la tabla `transactions_clean` ya tiene los IDs de producto perfectamente organizados en una lista (Array). Ahora toca explotar esta estructura.

Objetivo: Crea la tabla `sprint3_gold.product_sales_ranking` que contenga el inventario completo de productos y cuántas veces se ha vendido cada uno.

Requisitos del Informe:

- Detalle del Producto: Ha de incluir `product_id`, `name`, `price` y `color` (vienen de la tabla `products_clean`).
- Métrica de Negocio: Una columna nueva `total_sold` que cuente cuántas veces aparece este producto en las transacciones.
- Integridad: Han de aparecer todos los productos, incluso los que tienen 0 ventas (puede ser necesario descatalogarlos).

Pista Técnica:

- En `transactions_clean`, los productos están “encapsulados” dentro de un Array para cada transacción.
- Primero necesitarás “aplanar” la tabla de transacciones usando `UNNEST(product_ids)` para poder contar los productos individualmente.
- Después, has un cruce (`LEFT JOIN`) comenzando por la tabla de productos para no perder aquellos que no se han vendido nunca.

=====

Al principio se usa `LEFT JOIN` como los ejercicios anteriores para no perder productos que nunca se han vendido. Realizamos una subconsulta para poder “aplanar” los arrays y es en el `JOIN` donde se aplica `UNNEST(product_ids)` para desempaquetar cada array de productos en filas separadas y después conectarlas con sus id correspondientes.

Sin embargo, se reescribió utilizando una CTE (Common Table Expression) mediante WITH, que separa la lógica de "aplanar" los arrays de productos en un bloque independiente y con nombre (product_sales) antes de la consulta principal.

Ambas versiones son funcionalmente equivalentes y tienen el mismo coste de procesamiento, pero la versión con CTE de acuerdo a bibliografía, mejora la legibilidad del código

```

1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_gold.product_sales_ranking`
2 AS
3 WITH product_sales AS (
4   SELECT transaction_id, product_id
5   FROM `sprint3-analytics-yonamanrique.sprint3_silver.transactions_clean`,
6   UNNEST(product_ids) AS product_id
7 )
8 SELECT
9   p.product_id,
10  p.name,
11  p.price,
12  p.colour,
13  COUNT(ps.transaction_id) AS total_sold
14 FROM `sprint3-analytics-yonamanrique.sprint3_silver.products_clean` AS p
15 LEFT JOIN product_sales AS ps ON ps.product_id = p.product_id
16 GROUP BY p.product_id, p.name, p.price, p.colour
17 ORDER BY total_sold DESC;

```

Se completó la consulta

```

1 SELECT *
2 FROM `sprint3-analytics-yonamanrique.sprint3_gold.product_sales_ranking`;

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes		Resultados de la consulta					Chat	View all jobs	Guardar los resultados
3:53 p.m.		Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución		
Fila	product_id	name	price	colour	total_sold				
1	52	riverlands the duel	31.52	#727272	2654				
2	29	Tully maester Tully	167.2	#111111	2635				
3	21	duel Direwolf	96.9	#e2e2e2	2609				
4	16	the duel warden	180.91	#666666	2608				
5	66	mustafar jinn	2.12	#545454	2601				
6	87	sith Jade	96.26	#e2e2e2	2598				
7	33	duel warden	127.09	#191919	2597				
8	48	rock Renly in	21.53	#9e9e9e	2597				
9	23	riverlands north	169.96	#545454	2593				
10	68	Stark Karstark	167.37	#333333	2589				
11	88	Stannis warden south	167.15	#515151	2587				
12	72	Dance of the Dragons	114.00	#808080	2584				

Resultados por página: 50 1 - 50 de 100

Este informe incluye:

Detalle del producto: En el SELECT incluimos p.product_id, p.name, p.price y p.colour que vienen de products_clean.

Métrica de Negocio: COUNT(t.product_id) AS total_sold cuenta cuántas veces aparece cada producto en las transacciones después de hacer el UNNEST. Cada vez que un producto aparece en una transacción suma 1.

Integridad: Empezamos desde products_clean y hacemos LEFT JOIN con las transacciones, lo que significa que aunque un producto no tenga ninguna venta, aparece igualmente en el resultado con total_sold = 0. Si hubiéramos usado un JOIN normal, los productos sin ventas desaparecerían del resultado y no sabríamos que existen.

Ex. 3: Exportación de Resultados

Tu manager no tiene acceso a BigQuery y quiere el listado de "Top Products" en un Excel para una reunión.

Tarea: Exporta los datos de la tabla `product_sales_ranking` a Google Sheets o baja el archivo CSV localmente.

Entrega: Una captura de pantalla donde se vea el archivo abierto (Excel / Sheets) con los datos correctamente formateados.

=====

The screenshot shows the BigQuery console interface. At the top, there's a query editor with the following SQL query:

```
1 SELECT *  
2 FROM `sprint3-analytics-yonamanrique.sprint3_gold.product_sales_ranking`;
```

Below the query editor, a status message indicates the query is completed. The main area displays the query results in a table format:

File	product_id	name	price
1	52	riverlands the duel	
2	29	Tully maester Tarly	
3	21	duel Direwolf	

On the right side, the 'Export' menu is open, showing various options. The 'Guardar los resultados' (Save results) button is highlighted with a red box. The 'Descarga local' (Download locally) option is also highlighted with a red box, showing sub-options for CSV (up to 10 MB) and JSON (up to 10 MB).

Al no tener cuenta de facturación activa, no es posible exportar directamente a Google Cloud Storage. Como alternativa, se ejecuta un `SELECT *` sobre la tabla y se descarga el resultado como CSV local desde el editor de BigQuery, opción válida para ficheros de hasta 10 MB.

product_sales_ranking					
Visualización Zoom Añadir categoría Tabla de pivotación Insertar Tabla Gráfica Texto Fig					
+ Hoja 1					
A B C D E					
	product_sales_ranking				
1	product_id	name	price	colour	total_sold
2	52	riverlands the duel	31.52	#727272	2654
3	29	Tully maester Tarly	167.2	#111111	2635
4	21	duel Direwolf	96.9	#e2e2e2	2609
5	16	the duel warden	180.91	#666666	2608
6	66	mustafar jinn	2.12	#545454	2601
7	87	sith Jade	96.26	#e2e2e2	2598
8	33	duel warden	127.09	#191919	2597
9	48	rock Renly in	21.53	#9e9e9e	2597
10	23	riverlands north	169.96	#545454	2593
11	68	Stark Karstark	167.37	#333333	2589

Se comprueba que se ha descargado correctamente abriéndolo en una hoja de cálculo.

Referencias

1. <https://docs.cloud.google.com/bigquery/docs/external-tables?hl=es-419>
2. [https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language#create external table statement](https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language#create%20external%20table%20statement)
3. <https://docs.cloud.google.com/bigquery/docs/hive-partitioned-queries?hl=es-419>
4. <https://docs.cloud.google.com/bigquery/docs/best-practices-costs?hl=es-419>
5. <https://docs.cloud.google.com/bigquery/docs/views-intro?hl=es-419>
6. <https://cloud.google.com/blog/products/data-analytics/cost-optimization-best-practices-for-bigquery>